Technical Design Document: Tour CMF

This document describes the current implementation and its limitations. It is an as-built design.

Purpose and scope

Tour CMF supports public travel-package discovery, editorial publishing, traveller enquiries, AI-assisted CRM and consultant quotations. Staff work in role-specific dashboards; public visitors browse published material and submit enquiries.

Architecture

Browser → Next.js frontend → FastAPI backend → PostgreSQL. The backend also connects to an optional AI provider and local quote PDF storage.

Docker Compose runs frontend, backend, and db. The browser uses NEXT_PUBLIC_API_URL; server-side frontend requests use API_URL inside the Compose network. The backend exposes /health and /docs. PostgreSQL data is stored in the postgres-data volume. Quote PDFs currently live in the backend container filesystem, which is not durable across container recreation.

Technology stack

• Frontend: Next.js 15, React, TypeScript and CSS; standalone Node 22 container.

• Backend: Python 3.13, FastAPI, SQLAlchemy, Pydantic, Alembic, Uvicorn and ReportLab.

• Database: PostgreSQL 16 and psycopg.

• Authentication: JWT bearer tokens and Argon2 password hashing.

• AI: Gemini or an OpenAI-compatible HTTP endpoint, with rule-based fallback.

Data model

Table	Purpose and relationships
users	Staff identity, password hash, role and active flag; email is unique.
packages	Journey identity, status, published-version pointer and scheduled time.
package_versions	Numbered snapshots of package content, creator and timestamp; unique per package and version number. Content is serialized text.
seasonal_prices	Price, date range and basis attached to a package version.
enquiries	Customer details, package/version reference, assignment, status, acknowledgement and AI result JSON.
enquiry_notes, enquiry_replies	Staff-authored records linked to enquiries.
activity_logs	Actor, action, entity identifier, details and timestamp.
blog_posts	Article, category, cover, SEO fields, status and author. Related package slugs are stored as serialized text.
quotes	Traveller, amount, inclusions, optional package/enquiry references, consultant, public token and PDF path.
alembic_version	Applied migration revision.

The schema uses primary keys, foreign keys, unique slugs and enquiry references, and indexes on common status and relationship columns. Status values and several numeric/date invariants are not enforced by database check constraints. A published-version foreign key does not itself ensure that the version belongs to the same package. Related blog slugs are not foreign keys. Quote/package deletion semantics and migration downgrades should be hardened before production.

API design

The application API is prefixed /api/v1; FastAPI publishes request/response schemas at /docs.

Area	Routes	Access
Authentication	/auth/register, /auth/login, /auth/me, /auth/admin/check	Registration/login public; other routes authenticated/admin as applicable. Registration creates a consultant account.
Public catalogue	GET /packages, GET /packages/{slug}	Public; published packages.
Package CMS	/admin/packages, /{id}, /{id}/duplicate, /{id}/versions, /{id}/versions/{version_id}/restore	Admin/editor for CMS operations, with publication and destructive operations restricted by backend rules.
Blog	GET /blog, GET /blog/{slug}, /blog/admin and /blog/admin/{id}	Public reads of published posts; admin/editor manage posts.
Enquiries	POST /enquiries, GET /enquiries, PATCH /enquiries/{id}	Public creation; admin/consultant staff operations.
AI CRM	/enquiries/{id}/analyze, /analysis, /recommendations, /reply-draft and /reply-draft/approve	Admin/consultant.

Area	Routes	Access
Activity	GET /admin/activity	Authenticated admin/editor/consultant.
Quotes	POST /quotes, GET /quotes/{token}, GET /quotes/{token}/pdf	Creation by admin/consultant; token-bearing links are public.

Workflows

1. A staff member logs in; the frontend stores the bearer token in session storage and redirects by role. Backend dependencies validate the token and user role on protected routes.

2. Package saves create revisions with version-specific prices. A published-version pointer identifies public content. Scheduling is evaluated when the public catalogue is read, rather than by an independent timer.

3. Blog writers save drafts or publish posts; public blog reads select published posts.

4. An enquiry is saved and assigned a reference, and a fixed acknowledgement is recorded. The AI pipeline runs after the initial commit; errors are logged without rolling back the enquiry. The acknowledgement is displayed/recorded, not delivered by email or WhatsApp.

5. Consultants review extracted fields and drafts in the CRM. Approval records a reply; no external delivery integration is implied by the stored reply.

6. A consultant creates a quote; the backend generates a PDF and tokenized public link. The current builder accepts an entered amount, so quote pricing is not automatically guaranteed to come from the package price table.

AI integration and safeguards

The pipeline extracts travel requirements, selects candidate published packages and generates a reply draft. Package candidate prices are derived from seasonal prices. The LLM may be used for extraction or draft text; fallback rules provide results when it is unavailable. Structured AI output and drafts are saved in enquiry JSON fields and can be corrected or reviewed by staff. Since generated prose is not mechanically validated against the price table, a human must verify any price in a draft before use. Configuring an external provider may transmit enquiry content to it.

Security and operational considerations

• Authorization is enforced by backend role dependencies; client redirects are navigation aids, not security boundaries.

• JWTs are held in browser session storage; access tokens expire according to configuration. CORS is an origin allowlist, not authentication.

• Public registration currently creates consultant users; restrict or disable this route before public deployment if self-registration is not intended.

• Quote URLs are bearer-style secrets. Anyone with the token can view the quote; there is no expiry or revocation flow in the current model.

• Protect real credentials, use HTTPS and a strong signing secret, restrict database exposure, and configure backups and access controls.

• Audit logs cover implemented actions but are not a complete security audit trail. Avoid placing sensitive data in logs.

Performance Optimization and 4G Loading Requirements

Performance Target: All major customer-facing pages should achieve a loading performance target of under 2.5 seconds on a simulated mobile 4G network.

Initial Performance Assessment

An initial Lighthouse performance assessment identified that the homepage and package detail page were not meeting the required performance target.

Page	Measured LCP	Target LCP	Status
Homepage	6.43 seconds	Under 2.5 seconds	Needs optimization
Package Detail	5.71 seconds	Under 2.5 seconds	Needs optimization

LCP (Largest Contentful Paint) measures how long it takes for the largest visible content element to render.

Identified Performance Bottlenecks

The initial assessment identified the following major contributors to slow page loading:

1. Large Hero Images: Oversized hero images increase image download time and delay the rendering of the largest visible content element.

2. Externally Loaded Fonts: External font requests introduce additional network dependencies and can delay text rendering.

Optimization Strategy

The following improvements are planned to address the identified bottlenecks:

• Optimize hero images using modern formats such as WebP or AVIF.

• Resize and compress images according to their display dimensions.

• Implement responsive image loading using appropriate image sizes.

• Prioritize loading of above-the-fold and LCP images.

• Use lazy loading for below-the-fold images.

• Reduce external font dependencies by using locally hosted fonts where appropriate.

• Preload critical fonts and use appropriate font-display settings.

• Review unnecessary render-blocking resources.

Validation and Performance Verification

After implementing the optimizations, rebuild the application and rerun Lighthouse tests using mobile device emulation and simulated 4G network conditions.

The homepage and package detail page will be prioritized for retesting against the 2.5-second LCP target.

Final performance results will be documented after optimization, including the measured LCP values and remaining bottlenecks.

Deployment Consideration: Actual production performance may vary depending on hosting infrastructure, server response times, CDN configuration, image delivery, device capabilities, and network conditions. Therefore, deployed performance should also be monitored after release.